

CHAPTER 2: PROJECT PLANNING & ARCHITECTURE SETUP

2.1 Introduction to Project Planning

2.2 Project Objectives and Scope

2.3 Requirement Analysis

2.4 Feasibility Study

2.5 Technology Selection and Justification

2.6 Software Architecture Design

2.7 Project Folder Structure

2.8 Data Flow Architecture

2.9 API Planning and Strategy

2.10 Security Planning and Implementation

2.11 Scalability and Future Expansion

2.12 Benefits of Proper Project Planning

2.13 Risk Assessment and Mitigation

2.14 Chapter Summary

2.1 Introduction to Project Planning

Project planning and architecture setup form the backbone of every successful software application. Before implementation, it is necessary to define project scope, identify objectives, analyze user requirements, select suitable technologies, and design scalable architecture.

Proper planning minimizes development risks, improves productivity, reduces maintenance costs, and ensures all components work together efficiently. In financial technology applications like FinWise AI, where multiple technologies interact simultaneously, project planning becomes critical.

Importance of Project Planning

Aspect	Importance
Risk Management	Identifies potential issues before they occur
Resource Optimization	Allocates resources efficiently
Time Management	Provides realistic timelines and milestones
Cost Control	Prevents budget overruns
Quality Assurance	Ensures deliverables meet standards
Stakeholder Alignment	Ensures all stakeholders share the same vision

Project Planning Phases

Phase	Activities	Outputs
Inception	Ideation, feasibility analysis, initial planning	Project charter, feasibility report
Requirements	User needs analysis, feature definition	Requirements document
Architecture	Technology selection, system design	Architecture document, design specs
Implementation	Development, testing, integration	Working application
Deployment	Release, monitoring, maintenance	Deployed application

2.2 Project Objectives and Scope

Primary Objectives

- Intelligent Financial Advisory:** Develop an application that provides AI-powered financial recommendations and insights.
- Loan Eligibility Assessment:** Simplify and automate the process of checking loan eligibility.
- Credit Analysis:** Provide comprehensive credit score analysis and improvement suggestions.
- EMI Calculation:** Enable users to calculate and understand their monthly repayment obligations.

- 5. **Risk Assessment:** Evaluate financial risk associated with loan applications.
- 6. **Cloud Integration:** Implement secure cloud-based storage using Google Sheets.
- 7. **User Experience:** Provide an attractive, responsive, and user-friendly interface.
- 8. **AI Demonstration:** Showcase practical AI integration in modern web applications.
- 9. **Software Engineering:** Follow best practices including modularity, reusability, and scalability.

Secondary Objectives

Objective	Description
Financial Education	Help users understand financial concepts
Data Analysis	Enable data-driven decision making
Accessibility	Make financial services accessible to all
Cost Reduction	Reduce dependency on traditional financial advisors
Time Savings	Provide instant financial insights

Project Scope

In-Scope:

Feature	Description
Loan Eligibility Checker	Automated eligibility assessment based on user inputs
Credit Score Analyzer	Classification and recommendations for credit scores
EMI Calculator	Monthly EMI computation with amortization schedule
Risk Assessment	Financial risk evaluation and classification
AI Recommendations	Personalized financial advice from Claude AI
Cloud Storage	Google Sheets integration for record keeping
Dashboard	Interactive dashboard with results visualization
Responsive UI	Mobile-first, responsive user interface

Out-of-Scope:

Feature	Reason
Real-time Banking API Integration	Future enhancement, not in current scope
User Authentication	Future enhancement, prototype phase
Mobile Application	Future enhancement, currently web-only

Feature	Reason
Payment Processing	Not required for assessment tool
Multi-currency Support	Future enhancement, currently INR only
Historical Data Analysis	Future enhancement, prototype phase

2.3 Requirement Analysis

Functional Requirements

User Input Requirements:

Requirement ID	Description	Priority
FR-01	Users must be able to enter monthly salary	High
FR-02	Users must select employment type	High
FR-03	Users must enter age	High
FR-04	Users must enter credit score	High
FR-05	Users must enter existing EMI amount	High
FR-06	Users must enter requested loan amount	High
FR-07	Users must enter desired loan tenure	High

Processing Requirements:

Requirement ID	Description	Priority
FR-08	System must validate all user inputs	High
FR-09	System must calculate loan eligibility	High
FR-10	System must analyze credit score	High
FR-11	System must compute EMI	High
FR-12	System must assess financial risk	High
FR-13	System must generate loan recommendations	Medium

Output Requirements:

Requirement ID	Description	Priority
FR-14	System must display eligibility status	High

Requirement ID	Description	Priority
FR-15	System must show credit score classification	High
FR-16	System must display EMI calculation results	High
FR-17	System must provide AI recommendations	Medium
FR-18	System must show risk assessment results	Medium
FR-19	System must display dashboard with results	High

Storage Requirements:

Requirement ID	Description	Priority
FR-20	System must store user data in Google Sheets	Medium
FR-21	System must maintain data consistency	Medium
FR-22	System must ensure data privacy	High

Non-Functional Requirements

Performance Requirements:

Requirement ID	Description	Target
NFR-01	Page load time	< 3 seconds
NFR-02	API response time	< 5 seconds
NFR-03	Calculation time	< 1 second
NFR-04	Dashboard update time	< 2 seconds

Security Requirements:

Requirement ID	Description	Target
NFR-05	HTTPS encryption	Mandatory
NFR-06	Input validation	All inputs
NFR-07	API key protection	Secure storage
NFR-08	Data sanitization	All data

Reliability Requirements:

Requirement ID	Description	Target
NFR-09	System uptime	99.9%

Requirement ID	Description	Target
NFR-10	Error recovery	Graceful degradation
NFR-11	Data backup	Automatic
NFR-12	Error logging	Comprehensive

Usability Requirements:

Requirement ID	Description	Target
NFR-13	Responsive design	All screen sizes
NFR-14	Accessibility	WCAG 2.1 AA
NFR-15	Intuitive navigation	User-friendly
NFR-16	Clear feedback	Real-time

2.4 Feasibility Study

Technical Feasibility

Aspect	Assessment	Details
Technology Availability	✓ Confirmed	All selected technologies are readily available
Browser Compatibility	✓ Confirmed	Works on all modern browsers
API Availability	✓ Confirmed	Claude AI API accessible
Cloud Services	✓ Confirmed	Google Sheets and Apps Script available
Development Tools	✓ Confirmed	All tools are free and accessible
Integration Feasibility	✓ Confirmed	All components can be integrated

Economic Feasibility

Aspect	Assessment	Details
Development Cost	✓ Confirmed	All development tools are free
API Usage Cost	✓ Confirmed	Claude AI subscription manageable
Cloud Storage Cost	✓ Confirmed	Google Sheets free tier sufficient
Infrastructure Cost	✓ Confirmed	Minimal hosting requirements
Maintenance Cost	✓ Confirmed	Low maintenance overhead

Aspect	Assessment Details
ROI Projection	✓ Confirmed Positive ROI potential
Operational Feasibility	
Aspect	Assessment Details
Ease of Use	✓ Confirmed User-friendly interface design
Technical Skills	✓ Confirmed Standard web development skills
Training Requirement	✓ Confirmed Minimal training needed
User Adoption	✓ Confirmed Low barrier to entry
Maintenance	✓ Confirmed Easy maintenance and updates
Support	✓ Confirmed Standard support requirements
Legal Feasibility	
Aspect	Assessment Details
Data Privacy	✓ Confirmed Complies with privacy regulations
API Terms	✓ Confirmed Complies with Claude AI terms
Content Usage	✓ Confirmed Appropriate content usage
Intellectual Property	✓ Confirmed No IP violations
Regulatory Compliance	✓ Confirmed Follows standard practices

2.5 Technology Selection and Justification

Technology Selection Criteria

Criteria	Weight	Description
Compatibility	High	Works well with other selected technologies
Maturity	High	Stable and well-documented
Community Support	Medium	Active community and resources
Learning Curve	Medium	Reasonable learning requirements
Performance	High	Meets performance requirements
Cost	Medium	Cost-effective solution

Criteria	Weight	Description
Scalability	Medium	Supports future growth
Security	High	Meets security requirements

Technology Justification

HTML5:

- Industry standard for web content
- Semantic elements improve accessibility
- Native form validation capabilities
- Excellent browser support
- Future-proof technology

CSS3:

- Modern design capabilities
- Flexbox and Grid for responsive layouts
- CSS variables for maintainability
- Animation and transition support
- Glassmorphism implementation

JavaScript (ES6+):

- Universal web language
- Excellent ecosystem and libraries
- Asynchronous programming support
- Strong community and documentation
- Financial calculation capabilities

Claude AI API:

- Advanced natural language processing
- Context-aware understanding
- High-quality recommendations
- Regular model updates
- Good documentation and support

Google Sheets:

- Free and accessible

- Easy to set up and use
- Automatic backup and sync
- Familiar interface
- No complex database setup

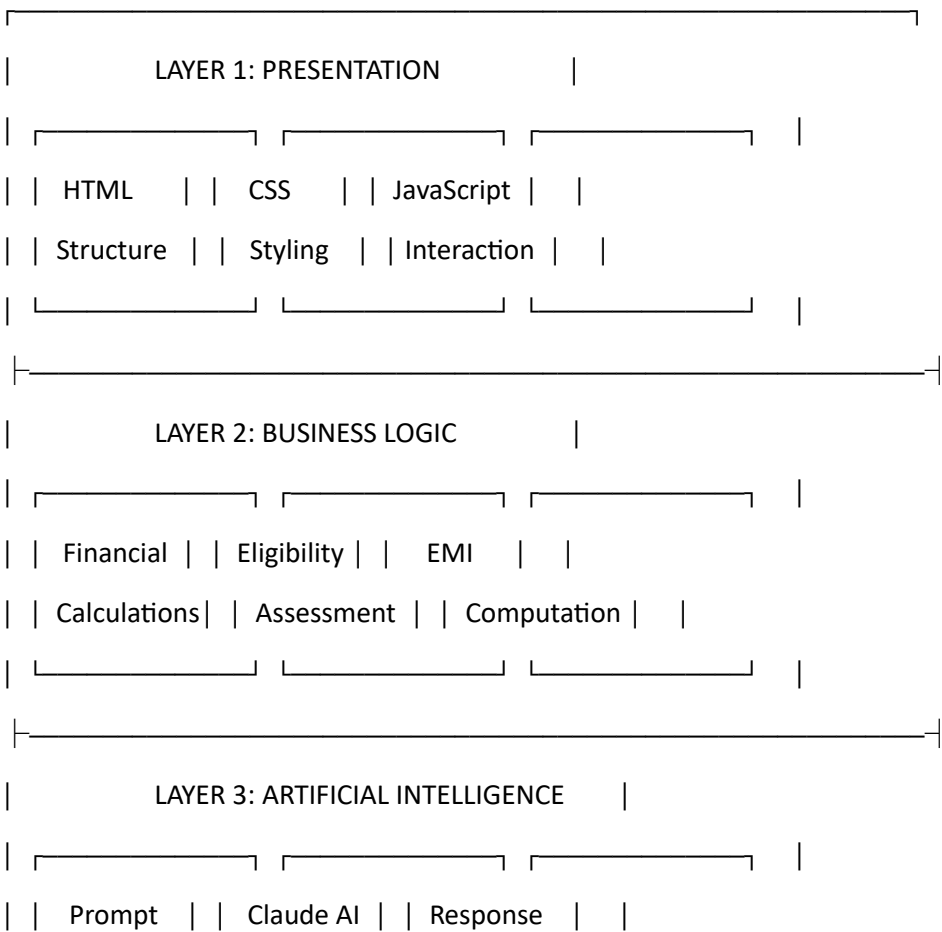
Google Apps Script:

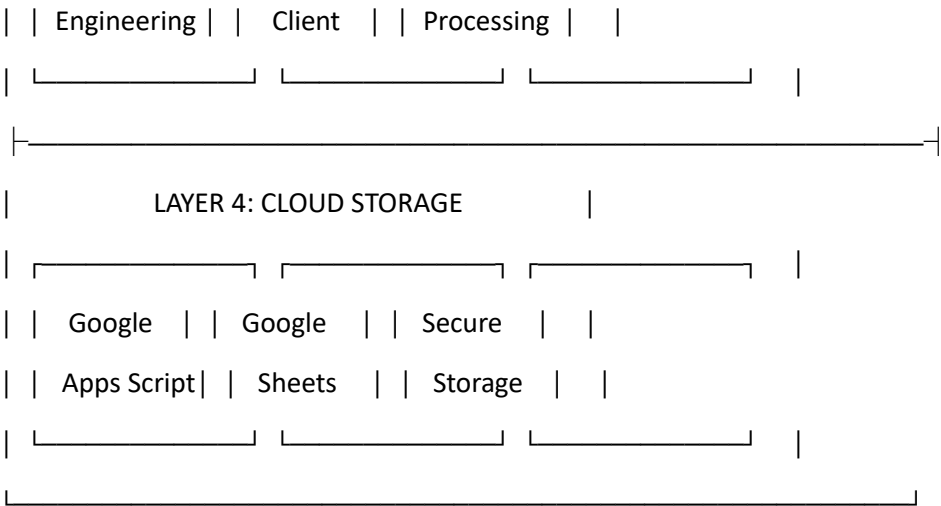
- Seamless Google Sheets integration
- Secure middleware layer
- Good documentation
- Free to use
- Easy deployment

2.6 Software Architecture Design

Architectural Overview

The FinWise AI platform follows a modular layered architecture with clear separation of concerns:
text





Architecture Components

Component	Description	Technology
User Interface	Visual presentation and user interaction	HTML, CSS, JavaScript
Application Logic	Business rules and calculations	JavaScript
Financial Engine	Financial calculations and assessments	JavaScript
AI Integration	Natural language processing	Claude AI API
Data Layer	Data storage and retrieval	Google Sheets, Apps Script
API Gateway	External API communication	Fetch API

Architectural Principles

Principle	Application
Separation of Concerns	Each layer has distinct responsibility
Modularity	Components can be developed and maintained independently
Reusability	Code can be reused across modules
Scalability	Architecture supports future growth
Security	Security implemented at all layers
Maintainability	Clean, well-documented code

2.7 Project Folder Structure

Complete Folder Structure

text

FinWise-AI/

```
|
|
| └─ index.html          # Main entry point
| └─ privacy.html        # Privacy policy
| └─ terms.html          # Terms of service
|
| └─ css/
|   | └─ style.css        # Main stylesheet
|   | └─ responsive.css   # Responsive design styles
|   | └─ animations.css   # Animation styles
|   | └─ glassmorphism.css # Glassmorphism theme
|   | └─ dashboard.css    # Dashboard specific styles
|   | └─ forms.css        # Form styling
|   |
|   | └─ js/
|   |   | └─ main.js       # Main application logic
|   |   | └─ financial.js  # Financial calculations
|   |   | └─ validation.js # Input validation
|   |   | └─ api.js        # API communication
|   |   | └─ dashboard.js  # Dashboard updates
|   |   | └─ eligibility.js # Eligibility assessment
|   |   | └─ emi.js        # EMI calculator
|   |   | └─ credit-score.js # Credit score analysis
|   |   | └─ risk-assessment.js # Risk assessment
|   |   | └─ recommendations.js # AI recommendations
|   |   | └─ storage.js    # Google Sheets integration
|   |   | └─ ui.js         # UI utilities
|   |   | └─ animations.js # Animation controls
|   |   | └─ events.js     # Event handlers
|   |   | └─ utils.js      # Utility functions
```

```
|
|
| └─ assets/
|   |
|   | └─ images/
|   |   |
|   |   | └─ logo.svg          # Application logo
|   |   |
|   |   | └─ hero-bg.jpg       # Hero background
|   |   |
|   |   | └─ icons/           # Icon files
|   |   |   |
|   |   |   | └─ illustrations/ # Illustration files
|   |   |
|   |   | └─ fonts/
|   |   |   |
|   |   |   | └─ main-font.woff2 # Primary font
|   |   |   |
|   |   |   | └─ heading-font.woff2 # Heading font
|   |   |
|   |   | └─ icons/
|   |   |   |
|   |   |   | └─ favicon.ico     # Browser favicon
|   |   |   |
|   |   |   | └─ apple-touch-icon.png # Mobile icon
|   |   |   |
|   |   |   | └─ svg/           # SVG icons
|   |   |
|   |   | └─ sounds/           # Sound files (if needed)
|   |
|   |
|   | └─ docs/
|   |   |
|   |   | └─ README.md          # Project documentation
|   |   |
|   |   | └─ CONTRIBUTING.md     # Contribution guidelines
|   |   |
|   |   | └─ API.md             # API documentation
|   |   |
|   |   | └─ user-guide.md       # User guide
|   |   |
|   |   | └─ technical-specs.md  # Technical specifications
|   |
|   |
|   | └─ config/
|   |   |
|   |   | └─ .env                # Environment variables
|   |   |
|   |   | └─ .gitignore          # Git ignore file
|   |   |
|   |   | └─ .prettierrc         # Prettier configuration
|   |   |
|   |   | └─ .eslintrc.js        # ESLint configuration
|   |   |
|   |   | └─ .husky/            # Husky configuration
|   |
|   |
|   |
```

```

├── scripts/
|   ├── deploy.sh          # Deployment script
|   ├── backup.sh         # Backup script
|   └── test.sh           # Testing script
|
├── tests/
|   ├── unit/             # Unit tests
|   ├── integration/      # Integration tests
|   └── e2e/             # End-to-end tests
|
├── google-apps-script/
|   ├── Code.gs           # Main Apps Script code
|   ├── validation.gs     # Validation functions
|   ├── storage.gs        # Storage functions
|   └── utilities.gs      # Utility functions
|
├── prompts/
|   ├── eligibility-prompts.txt # Eligibility prompts
|   ├── recommendation-prompts.txt # Recommendation prompts
|   └── improvement-prompts.txt # Improvement suggestions
|
└── .github/
    └── workflows/
        └── ci.yml        # CI/CD workflow

```

Folder Descriptions

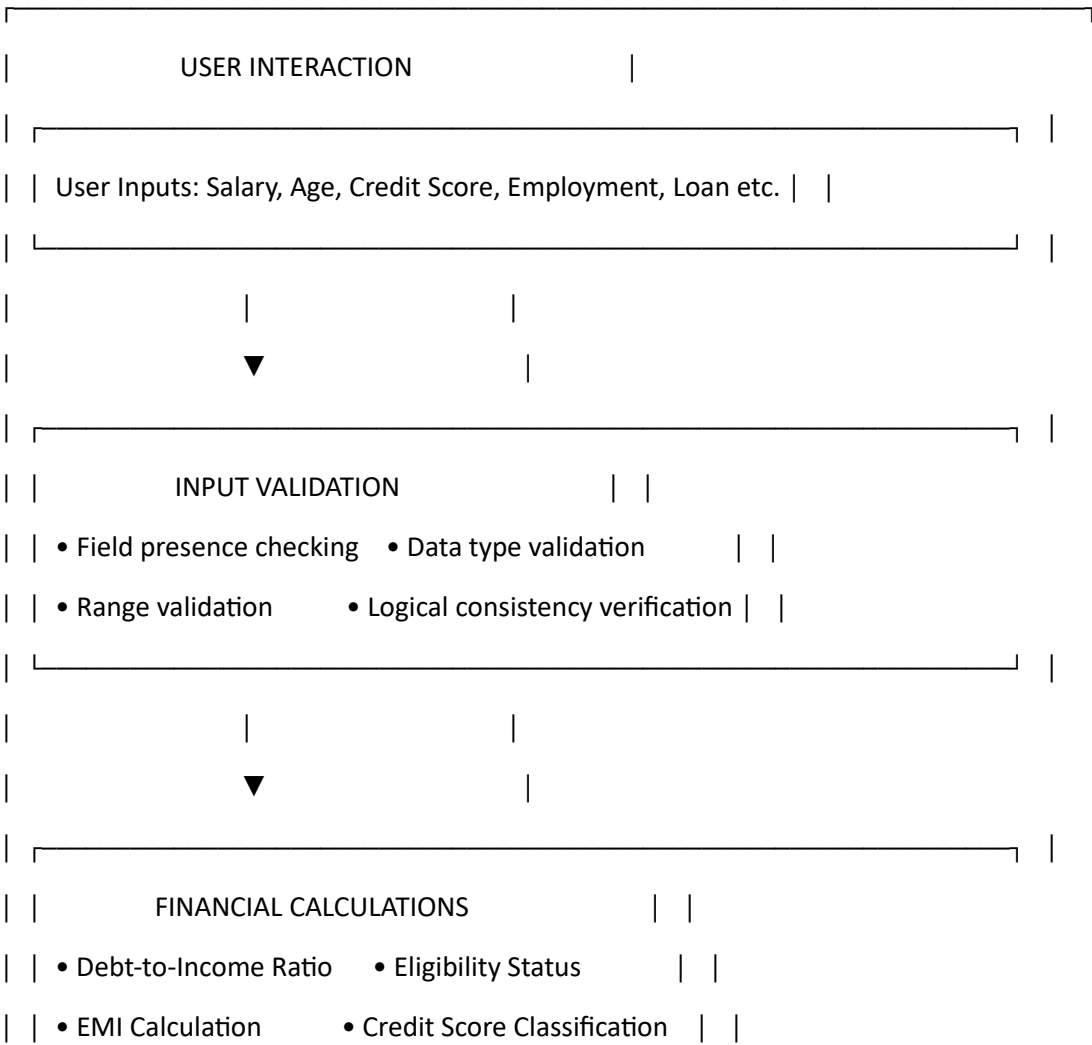
Folder	Purpose
css/	All CSS styles organized by function
js/	All JavaScript modules organized by feature
assets/	Static assets including images, fonts, and icons

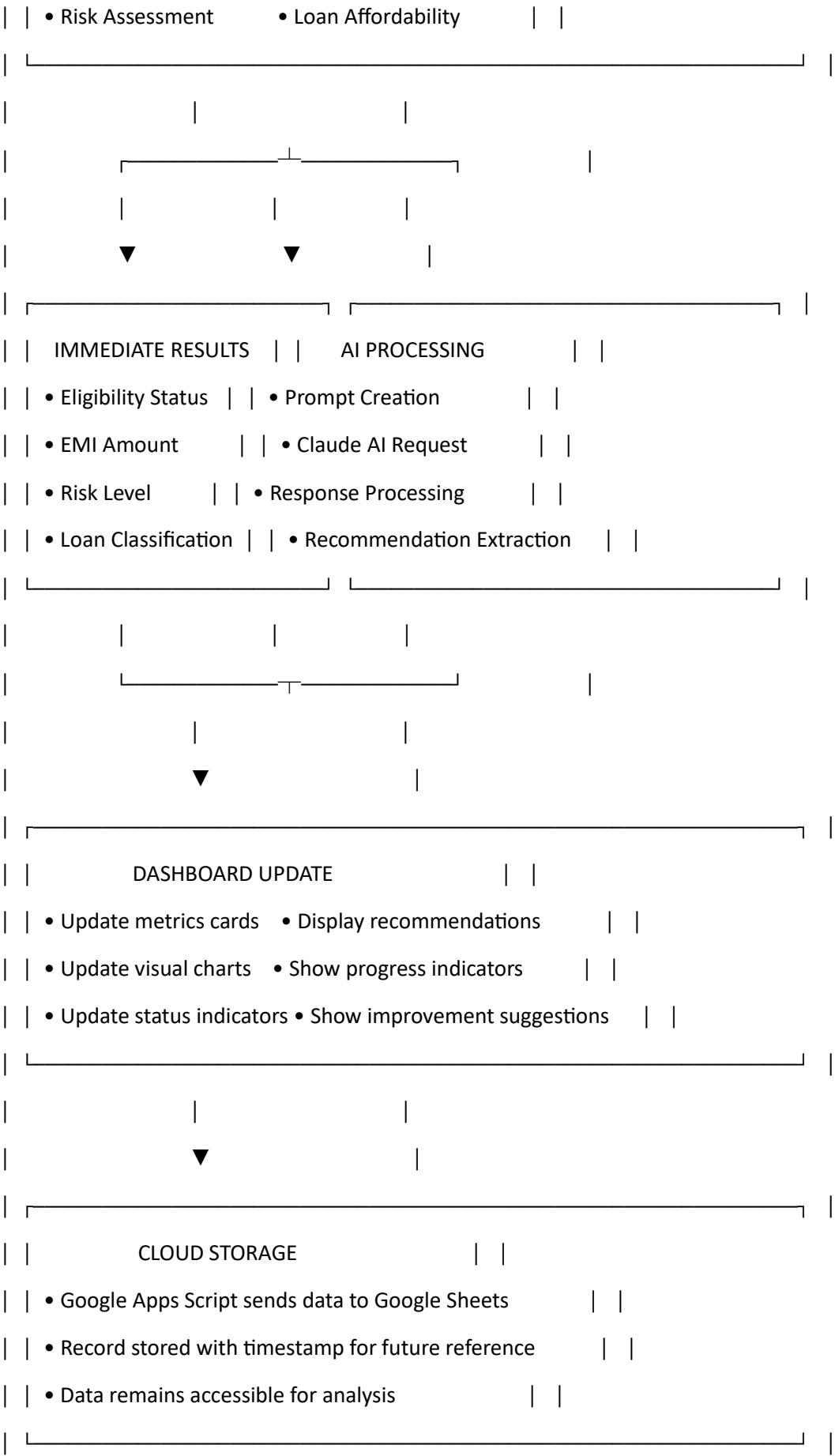
Folder	Purpose
docs/	Project documentation and guides
config/	Configuration files and environment variables
scripts/	Deployment and utility scripts
tests/	Unit, integration, and end-to-end tests
google-apps-script/	Google Apps Script code for cloud integration
prompts/	AI prompt templates and examples
.github/	GitHub configuration and workflows

2.8 Data Flow Architecture

Complete Data Flow Diagram

text





Data Processing Stages

Stage	Processing Steps	Output
Stage 1: Input Collection	- User enters data - Data captured via forms - Real-time validation	Validated user inputs
Stage 2: Preliminary Analysis	- Basic financial calculations - Eligibility screening - Initial risk assessment	Preliminary results
Stage 3: AI Processing	- Prompt generation - Claude AI request - Response analysis	AI recommendations
Stage 4: Result Generation	- Combine all results - Format for display - Update dashboard	Complete results
Stage 5: Storage	- Package data - Send to Apps Script - Store in Sheets	Persistent record

2.9 API Planning and Strategy

Claude AI API Integration

Request Structure:

```
json
{
  "model": "claude-3-opus-20240229",
  "messages": [
    {
      "role": "user",
      "content": "Prompt with financial information"
    }
  ],
  "max_tokens": 1024,
  "temperature": 0.7
}
```


Response Handling:

Aspect	Implementation
Parsing	Extract recommendations from response
Formatting	Format for dashboard display
Error Handling	Handle API errors gracefully
Rate Limiting	Implement request queuing
Retry Logic	Retry failed requests

Google Apps Script API

Endpoint Configuration:

Aspect	Value
Endpoint	Deployed Apps Script Web App URL
Method	POST for storage, GET for retrieval
Content-Type	application/json
Authentication	OAuth or deployment token

Request Validation:

Validation	Implementation
Required Fields	Check all required fields present
Data Types	Verify correct data types
Value Ranges	Validate numerical ranges
Format	Check data format consistency

API Communication Protocol

Request Flow:

1. **Prepare Request:** Format data as JSON
2. **Set Headers:** Content-Type, Authentication
3. **Send Request:** HTTP POST via Fetch API
4. **Handle Response:** Parse JSON, handle errors
5. **Update UI:** Display results or error messages

Error Handling Strategy:

	• Data Minimization	• Access Control	
	• Data Anonymization	• Audit Logging	
	LAYER 4: API SECURITY		
	• API Key Management	• Rate Limiting	
	• Token Authentication	• Request Validation	
	• API Auditing	• Secure Communication	

Security Implementation Details

Input Validation:

Technique	Implementation
Client-Side	JavaScript validation with immediate feedback
Server-Side	Google Apps Script validation
Sanitization	Escape special characters
Type Checking	Ensure correct data types
Range Checking	Validate numerical ranges

Data Protection:

Measure	Implementation
Encryption	HTTPS for all communications
API Keys	Environment variables, not in code
Sensitive Data	Minimize collection and storage
Session Management	Secure session handling
Error Messages	Non-revealing error messages

Access Control:

Control	Implementation
Authentication	Optional OAuth for administrative functions
Authorization	Role-based access where applicable
Rate Limiting	Prevent API abuse
Monitoring	Log access attempts and errors

2.11 Scalability and Future Expansion

Scalability Considerations

Aspect	Current Implementation	Future Scalability
User Base	Single-user prototype	Multi-user with authentication
Data Storage	Google Sheets	SQL database or cloud storage
API Calls	Individual requests	Batch processing, caching
Infrastructure	Static hosting	Cloud hosting with auto-scaling
Features	Core financial features	Full suite of financial services

Future Enhancement Opportunities

Short-term Enhancements (0-6 months):

Enhancement	Description
User Authentication	Login and profile management
Historical Data Tracking	Track user history and progress
Multi-language Support	Support for regional languages
PDF Report Generation	Generate downloadable reports
Export Functionality	Export data to other formats

Medium-term Enhancements (6-12 months):

Enhancement	Description
Banking API Integration	Real-time banking data
Investment Planning	Portfolio recommendations
Tax Planning	Tax estimation and planning

Enhancement	Description
Insurance Advisory	Insurance recommendations
Goal Tracking	Financial goal monitoring

Long-term Enhancements (12+ months):

Enhancement	Description
Machine Learning	Predictive models for loan approval
Mobile Application	Native iOS and Android apps
Blockchain Integration	Secure transaction verification
Peer-to-Peer Lending	Platform for P2P lending
Comprehensive Advisory	Full-service financial advisory

2.12 Benefits of Proper Project Planning

Development Efficiency

Benefit	Impact
Reduced Development Time	Clear roadmap prevents delays
Improved Team Productivity	Clear roles and responsibilities
Better Resource Allocation	Optimal use of resources
Parallel Development	Independent module development
Reduced Rework	Proper planning reduces mistakes

Quality Assurance

Benefit	Impact
Improved Quality	Well-planned features work correctly
Simplified Testing	Clear test cases and scenarios
Better Maintainability	Organized, documented code
Reduced Risks	Risks identified and mitigated

Long-term Success

Benefit	Impact
Scalability	Foundation for growth
Team Collaboration	Shared understanding and goals
Reliability	Stable, dependable application
Continuous Improvement	Easy to add enhancements

2.13 Risk Assessment and Mitigation

Risk Identification

Risk ID	Risk	Probability	Impact	Mitigation
R-01	API Rate Limiting	Medium	High	Implement caching and batching
R-02	Data Privacy Breach	Low	High	Strong encryption and access control
R-03	Technical Issues	Medium	Medium	Comprehensive testing
R-04	Integration Failures	Medium	High	Thorough integration testing
R-05	User Adoption	Low	Medium	User-friendly design
R-06	Performance Issues	Low	Medium	Performance optimization

Risk Mitigation Strategies

Risk	Mitigation Strategy
API Rate Limiting	Implement request queuing, caching, and fallback mechanisms
Data Privacy Breach	Encrypt data, minimize collection, implement access control
Technical Issues	Comprehensive testing, error handling, logging
Integration Failures	Unit and integration testing, fallback options
User Adoption	User-centered design, clear value proposition
Performance Issues	Code optimization, caching, CDN usage

2.14 Chapter Summary

Key Takeaways

Area	Summary
Project Objectives	Clear goals for intelligent financial advisory system
Requirement Analysis	Comprehensive functional and non-functional requirements
Feasibility Study	Technical, economic, operational, and legal feasibility confirmed
Technology Selection	Appropriate technologies chosen for each layer
Software Architecture	Modular layered architecture designed
Folder Structure	Organized project directory layout
Data Flow	Clear workflow from input to output
API Planning	External service integration strategy
Security Planning	Comprehensive security measures
Scalability	Future expansion capabilities considered

Checklist

- Project objectives defined
- Requirements analyzed
- Feasibility confirmed
- Technologies selected
- Architecture designed
- Folder structure created
- Data flow documented
- API planning completed
- Security planning completed
- Scalability considered

Establishing a strong architecture before development ensures that every financial module functions efficiently while maintaining security, reliability, and future extensibility. A well-planned architecture not only simplifies implementation but also provides a solid foundation for integrating Artificial Intelligence into modern financial applications.